

Lecture 12: Tuples + Sets



INFO 1100: Introduction to Programming

time for
even more data structures

time for
even more data structures

but we know so many already!

time for
even more data structures

but we know so many already!

why these?

Tuples

Tuples

generalizes pairs, triples, quadruples

Tuples

Tuples

What are they?

Tuples

sequences of values
of any type

Tuples

How are they different
from lists?

Tuples

How are they different
from lists?

immutable

Tuples

are **immutable**

what does that mean?

Tuples

are **immutable**

fixed length

Tuples

are **immutable**

fixed length

hashable

can be used as
dictionary keys

Tuples

are **immutable**

fixed length

hashable

can be used as
dictionary keys

efficient

immutable
objects are
optimized

Tuples

are **immutable**
fixed length

hashable

can be used as
dictionary keys

efficient

immutable
objects are
optimized

conceptual

represent things
that shouldn't
change

Tuples

How are they different
from lists?

Tuples

How are they different
from lists?

heterogeneity?

Tuples

tuple syntax

`(<expr-1>, <expr-2>, ..., <expr-n>)`

Tuples

tuple syntax

`<expr-1>, <expr-2>, ..., <expr-n>`

or without parentheses!

Tuples

e.g.

(42 , 4 . 0 , ' x ')

Tuples

single elements in parentheses aren't tuples

Tuples

single elements in parentheses aren't tuples

```
type ( 42 )
```

Tuples

single elements in parentheses aren't tuples

```
type ( 42 )
```



```
<class 'int'>
```

Tuples

but an element followed by a comma is!

Tuples

but an element followed by a comma is!

$d = 1,$

Tuples

but an element followed by a comma is!

```
d = 1,
```

```
type(d) → <class 'tuple'>
```

Tuples

to create an empty tuple...

```
d = tuple()
```

Tuples

... or convert a string to a tuple...

`tuple('none')`  `('n', 'o', 'n', 'e')`

Tuples

you can use many list operators with tuples

Tuples

you can use many list operators with tuples

```
t = ( 'n' , 'o' , 'n' , 'e' )
```

Tuples

you can use many list operators with tuples

```
t = ( 'n' , 'o' , 'n' , 'e' )
```

```
t[1] → 'o'
```

Tuples

you can use many list operators with tuples

```
t = ( 'n' , 'o' , 'n' , 'e' )
```

```
t[1:] → ( 'o' , 'n' , 'e' )
```

Tuples

you can use many list operators with tuples

```
t = ( 'n' , 'o' , 'n' , 'e' )
```

```
t + ( 's' , )  $\longrightarrow$  ( 'n' , 'o' , 'n' , 'e' , 's' )
```

Tuples

you can use many list operators with tuples

```
t = ( 'n' , 'o' , 'n' , 'e' )
```

```
t * 2    →    ( 'n' , 'o' , 'n' , 'e' , 'n' ,  
                'o' , 'n' , 'e' )
```

Tuples

you can use many list operators with tuples

```
t = ( 'n' , 'o' , 'n' , 'e' )
```

```
sorted(t)  →  [ 'e' , 'n' , 'n' , 'o' ]
```

Tuples

you can use many list operators with tuples

```
t = ( 'n' , 'o' , 'n' , 'e' )
```

```
sorted(t)  →  [ 'e' , 'n' , 'n' , 'o' ]
```

returns a list!

Tuple **assignment**

you can use tuples to assign values to
multiple variables at once

Tuple **assignment**

you can use tuples to assign values to
multiple variables at once

```
<var-1>, <var-2>, ..., <var-n> = <len-n-tuple>
```

Tuple **assignment**

you can use tuples to assign values to multiple variables at once

```
first, last = 0, 6
```

Tuple **assignment**

you can assign variable names to items in a
sequence

Tuple **assignment**

you can assign variable names to items in a
sequence

```
one, two, three = "it is done".split()
```

Returning tuples

Returning tuples

used to return multiple values

Returning tuples

used to return multiple values

```
def min_max(lst):  
    return min(lst), max(lst)
```

Returning tuples

used to return multiple values

```
def min_max(lst):  
    return min(lst), max(lst)  
  
scores = [100, 82, 97]  
the_min, the_max = min_max(scores)
```

Tuples in **loops**

Tuples in **loops**

`enumerate(<seq>)` gives tuples
we can loop over

Tuples in **loops**

```
lst = ["Awake", "Brush", "Class"]  
  
for (idx, item) in enumerate(lst):  
    print(str(idx) + ". " + item)
```

Tuples in **loops**

```
lst = ["Awake", "Brush", "Class"]  
  
for (idx, item) in enumerate(lst):  
    print(str(idx) + ". " + item)
```



```
0. Awake  
1. Brush  
2. Class
```

Tuples

Enter in the chat:

When would you want to use a tuple?

Sets

Sets

Sets

What are they?

Sets

a collection of unique
elements

Sets

a collection of unique
elements

like the mathematical object

Sets

creating a set

Sets

creating a set

```
s = set()
```

Sets

creating a set

```
s = {1, 2, 3}
```

Sets

set membership

$s = \{1, 2, 3\}$

1 in s

Sets

set membership

```
s = {1, 2, 3}
```

```
1 in s
```



```
True
```

Sets

set membership

$s = \{1, 2, 3\}$

0 in s

Sets

set membership

`s = {1, 2, 3}`

`0 in s`



`False`

Sets

set membership

fast – items are hashed!

Sets

adding to sets

Sets

adding to sets

sets are mutable

Sets

adding to sets

sets are mutable

but items must be hashable

Sets

adding to sets

```
s = {1, 2, 3}
```

```
s.add(0)
```

Sets

adding to sets

`s = {1, 2, 3}`

`s.add(0)`

`s → {0, 1, 2, 3}`

Sets

adding to sets

```
s = {1, 2, 3}
```

```
s.add(1)
```

Sets

adding to sets

$s = \{1, 2, 3\}$

`s.add(1)`

$s \rightarrow \{1, 2, 3\}$

Sets

adding to sets

$s = \{1, 2, 3\}$

`s.add(1)`

$s \rightarrow \{1, 2, 3\}$

cannot have multiple copies

Sets

sets have no order!

$$s = \{1, 2, 3\}$$

Sets

sets have no order!

```
s = {1, 2, 3}
```

```
s.add(1024)
```

```
s.add(0)
```

Sets

sets have no order!

```
s = {1, 2, 3}
```

```
s.add(1024)
```

```
s.add(0)
```

```
s → {1024, 1, 2, 3, 0}
```

Sets

sets are iterable

Sets

sets are iterable

```
for item in {1, 2, 3}:  
    print(item)
```

Sets

you can create sets out of any
sequence

Sets

you can create sets out of any
sequence

```
set ( 'couscous' )
```

Sets

you can create sets out of any
sequence

```
set('couscous')
```



```
{ 'o', 'u', 's', 'c' }
```

Sets

you can create sets out of any
sequence

```
set([1, 1, 2, 3])
```

Sets

you can create sets out of any
sequence

```
set([1, 1, 2, 3])
```



```
{1, 2, 3}
```

Set **operations**

Set **operations**

union

Set **operations**

union

$s1 = \{ 'h', 'r', 'n' \}$

$s2 = \{ 'r', 'b' \}$

Set **operations**

union

`s1 = { 'h', 'r', 'n' }`

`s2 = { 'r', 'b' }`

`s1.union(s2)`

Set **operations**

union

`s1 = { 'h', 'r', 'n' }`

`s2 = { 'r', 'b' }`

`s1.union(s2)`



`{ 'h', 'r', 'n', 'b' }`

Set **operations**

union

`s1 = { 'h', 'r', 'n' }`

`s2 = { 'r', 'b' }`

`s1` | `s2`



`{ 'h', 'r', 'n', 'b' }`

Set **operations**

intersection

Set **operations**

intersection

$s1 = \{ 'h', 'r', 'n' \}$

$s2 = \{ 'r', 'b' \}$

Set **operations**

intersection

`s1 = { 'h', 'r', 'n' }`

`s2 = { 'r', 'b' }`

`s1.intersection(s2)`

Set **operations**

intersection

`s1 = { 'h', 'r', 'n' }`

`s2 = { 'r', 'b' }`

`s1.intersection(s2)`



`{ 'r' }`

Set **operations**

intersection

`s1 = { 'h', 'r', 'n' }`

`s2 = { 'r', 'b' }`

`s1 & s2`



`{ 'r' }`

Set **operations**

difference

$s1 = \{ 'h', 'r', 'n' \}$

$s2 = \{ 'r', 'b' \}$

Set **operations**

difference

`s1 = { 'h', 'r', 'n' }`

`s2 = { 'r', 'b' }`

`s1.difference(s2)`

Set **operations**

difference

`s1 = { 'h', 'r', 'n' }`

`s2 = { 'r', 'b' }`

`s1.difference(s2)`



`{ 'h', 'n' }`

Set **operations**

difference

$s1 = \{ 'h', 'r', 'n' \}$

$s2 = \{ 'r', 'b' \}$

$s1 - s2$



$\{ 'h', 'n' \}$

Set **operations**

equivalence

$s1 = \{ 'h', 'r', 'n' \}$

$s2 = \{ 'r', 'b' \}$

Set **operations**

equivalence

`s1 = { 'h', 'r', 'n' }`

`s2 = { 'r', 'b' }`

`s1 == s2`

Set **operations**

equivalence

`s1 = { 'h', 'r', 'n' }`

`s2 = { 'r', 'b' }`

`s1 == s2`



`False`

Set **operations**

subset

Set **operations**

subset

$s1 = \{ 'h', 'r', 'n' \}$

$s2 = \{ 'r' \}$

Set **operations**

subset

$s1 = \{ 'h', 'r', 'n' \}$

$s2 = \{ 'r' \}$

$s2 \leq s1$

Set **operations**

subset

`s1 = { 'h', 'r', 'n' }`

`s2 = { 'r' }`

`s2 <= s1`



`True`

Set **operations**

proper subset

`s1 = { 'h', 'r', 'n' }`

`s2 = { 'r' }`

`s2 < s1`



`True`

Set **operations**

superset

$s1 = \{ 'h', 'r', 'n' \}$

$s2 = \{ 'r' \}$

Set **operations**

superset

`s1 = { 'h', 'r', 'n' }`

`s2 = { 'r' }`

`s1 >= s2`

Set **operations**

superset

`s1 = { 'h', 'r', 'n' }`

`s2 = { 'r' }`

`s1 >= s2`



`True`

Set **operations**

proper superset

`s1 = { 'h', 'r', 'n' }`

`s2 = { 'r' }`

`s1 > s2`



`True`

Set

Enter in the chat:

When would you want to use a set?

Example 1

```
songs = [ ("Roar", "Katy Perry"),  
          ("Halo", "Beyonce"),  
          ("Firework", "Katy Perry") ]
```

```
artists = set()  
for title, artist in songs:  
    artists.add(artist)
```

```
print(len(artists))  
print(artists)
```

Example 2

```
sales = [("apple", 3), ("banana", 2),
          ("apple", 5), ("cherry", 1)]
totals = {}
for item, count in sales:
    if item in totals:
        total[item] += count
    else:
        total[item] = count

print(totals["apple"])
print(len(totals))
```

Example 3

```
enrolled = [ ("Ana", "CS"), ("Ben", "CS"),  
              ("Ana", "Math"), ("Cara", "Math") ]  
  
cs = set()  
math = set()  
for name, dept in enrolled:  
    if dept == "CS":  
        cs.add(name)  
    else:  
        math.add(name)  
  
print(cs.union(math))  
print(cs.intersection(math))
```

Example 4

```
def has_duplicates(items):  
    seen = set()  
    for x in items:  
        if x in seen:  
            return True  
        seen.add(x)  
    return False  
  
print(has_duplicates([1, 2, 3, 2]))  
print(has_duplicates([1, 2, 3, 4]))
```

Questions?

(... I'll wait for 2)

For tomorrow:

(07/16 @ 11:30am)

- Lab 12
- Readings
 - Think Python* – 14.1-14.4
 - “Defining a New Class of Object”
 - “Object Identity and Creating an Instance”
- GRQs